

Homework #3

Name(s): Raj Shah (ras637), (), ()

Problem 1: You are given an array of characters $S[1 : n]$ such that each $S[i]$ is a lowercase English letter. You also have a black-box algorithm $\text{dict}(i, j)$ that returns whether $S[i : j]$ is a valid English word in $O(1)$ time. Design and analyze a dynamic programming algorithm to decide whether S can be partitioned into a sequence of valid words. The runtime should be $O(n^2)$.

Answer

Use DP on prefixes. Let $dp[t]$ be true if $S[1 : t]$ can be split into valid words. Then $dp[t]$ is true if there exists $k < t$ such that $dp[k]$ is true and $\text{dict}(k + 1, t)$ is true.

Explanation

We only need to know where the last word starts. If the last word is $S[k + 1 : t]$, then the part before it must already be splittable. The dictionary check is $O(1)$, so trying all split points for each t is feasible. If any split works, we mark $dp[t]$ true. Finally, $dp[n]$ tells us whether the whole string is splittable.

Formula

$$dp[0] = \text{True}, \quad dp[t] = \bigvee_{k=0}^{t-1} (dp[k] \wedge \text{dict}(k + 1, t)) \quad \text{for } t = 1, \dots, n.$$

Proof of Correctness

Claim: $dp[t]$ is true if and only if $S[1 : t]$ can be partitioned into valid words.

If $dp[t]$ is true, then by the recurrence there exists a $k < t$ with $dp[k]$ true and $\text{dict}(k + 1, t)$ true. That means $S[1 : k]$ can be partitioned, and $S[k + 1 : t]$ is a valid word, so $S[1 : t]$ is partitionable.

If $S[1 : t]$ is partitionable, let the last word be $S[k + 1 : t]$. Then $S[1 : k]$ is also partitionable, so $dp[k]$ will be true, and $\text{dict}(k + 1, t)$ is true. Therefore the recurrence will set $dp[t]$ to true. So the claim holds.

Runtime Analysis

For each t , we try all $k = 0..t - 1$, so the total number of checks is

$$\sum_{t=1}^n t = O(n^2).$$

Each check calls dict in $O(1)$ time, so total runtime is $O(n^2)$ and space is $O(n)$.

Pseudo code

```
WordBreak(S[1..n]):  
    dp[0] = True  
    for t = 1 to n:  
        dp[t] = False  
        for k = 0 to t-1:  
            if dp[k] == True and dict(k+1, t) == True:  
                dp[t] = True  
                break  
    return dp[n]
```

Final Answer (boxed)

Compute $dp[t] = \bigvee_{k < t} (dp[k] \wedge dict(k+1, t))$ and return $dp[n]$. Time $O(n^2)$.

Problem 2: Suppose you are given an array $A[1 : n]$ of integers (positive or negative). A contiguous subarray $A[i : j]$ is called *positive* if $\sum_{k=i}^j A[k] > 0$. Design and analyze a DP algorithm that in $O(n^2)$ time finds the smallest number of positive contiguous subarrays such that every positive entry of A belongs to exactly one of these subarrays.

Answer

Use DP on prefixes with prefix sums. Let $dp[i]$ be the minimum number of chosen positive-sum subarrays needed to cover all positive entries in $A[1 : i]$ exactly once. If $A[i] \leq 0$, then $dp[i] = dp[i - 1]$. If $A[i] > 0$, then i must be inside the last chosen subarray $A[\ell : i]$ whose sum is positive, so we set $dp[i] = \min(dp[\ell - 1] + 1)$ over all valid ℓ .

Explanation

Every positive element must be covered, so when we reach a positive position i , we must close some subarray ending at i that has positive total sum. The subarray start ℓ can be anywhere earlier, so we try all ℓ values. Prefix sums let us test whether $A[\ell : i]$ is positive in $O(1)$ time. Taking the minimum over all valid starts gives the smallest number of subarrays.

Formula

Let $P[0] = 0$ and $P[i] = \sum_{k=1}^i A[k]$, so $\sum_{k=\ell}^i A[k] = P[i] - P[\ell - 1]$.

$$dp[0] = 0, \quad dp[i] = \begin{cases} dp[i - 1] & \text{if } A[i] \leq 0, \\ \min_{\ell \in [1, i] : P[i] - P[\ell - 1] > 0} (dp[\ell - 1] + 1) & \text{if } A[i] > 0. \end{cases}$$

Proof of Correctness

Claim: $dp[i]$ equals the minimum number of positive-sum subarrays that cover all positive entries in $A[1 : i]$ exactly once.

If $A[i] \leq 0$, then there is no new positive entry that must be covered at index i , so any optimal cover for $A[1 : i]$ is also an optimal cover for $A[1 : i - 1]$. Therefore $dp[i] = dp[i - 1]$ is correct.

If $A[i] > 0$, then index i must belong to exactly one chosen subarray, and we can assume the last chosen subarray ends at i for the prefix problem. Let that subarray be $A[\ell : i]$; it must have positive sum, and everything before $\ell - 1$ must be optimally covered with $dp[\ell - 1]$ subarrays. Thus any solution has value at least $dp[\ell - 1] + 1$ for some valid ℓ , and choosing the minimum over all valid ℓ gives the optimal value. So the recurrence produces the correct optimum.

Runtime Analysis

We compute prefix sums in $O(n)$. For each i , we may scan all $\ell \leq i$, so the total time is

$$\sum_{i=1}^n O(i) = O(n^2).$$

Space is $O(n)$ for P and dp .

Pseudo code

```
MinPositiveCover(A[1..n]):
    P[0] = 0
    for i = 1 to n:
        P[i] = P[i-1] + A[i]

    dp[0] = 0
    for i = 1 to n:
        if A[i] <= 0:
            dp[i] = dp[i-1]
        else:
            dp[i] = +infty
            for l = 1 to i:
                if P[i] - P[l-1] > 0:
                    dp[i] = min(dp[i], dp[l-1] + 1)
    return dp[n]
```

Final Answer (boxed)

Use prefix sums and $dp[i] = dp[i - 1]$ if $A[i] \leq 0$, $dp[i] = \min_{l: P[i] - P[l-1] > 0} (dp[l - 1] + 1)$. $O(n^2)$.

Problem 3: A number n is special if $n = a \cdot 197 + b \cdot 232$ for non-negative integers a, b . Write a DP algorithm $\text{Special}(n)$ that either finds such a, b or returns “no solution”.

Answer

We build DP for sums up to n . Let $dp[x]$ be true if x can be formed using 197 and 232. Then $dp[x]$ is true if $dp[x - 197]$ or $dp[x - 232]$ is true. We also store which step was used so we can reconstruct a and b by backtracking.

Explanation

This is like a coin-change reachability problem with coin sizes 197 and 232. We start from 0, which is always reachable. Whenever $x - 197$ is reachable, we can reach x by adding one 197, and similarly for 232. If we remember which coin was used to first reach each x , then when $dp[n]$ is true we can walk backward and count how many of each coin we used. If $dp[n]$ is false, then no combination exists.

Formula

$$dp[0] = \text{True}, \quad dp[x] = (x \geq 197 \wedge dp[x - 197]) \vee (x \geq 232 \wedge dp[x - 232]).$$

Proof of Correctness

Claim: $dp[x]$ is true if and only if there exist non-negative integers a, b such that $x = 197a + 232b$.

If $dp[x]$ is true, then the recurrence says we reached x from either $x - 197$ or $x - 232$. Repeating this argument back to 0 shows that x is a sum of some number of 197's and 232's, so $x = 197a + 232b$ for some $a, b \geq 0$.

If $x = 197a + 232b$, then if $a > 0$ we can write $x = (x - 197) + 197$ and $x - 197$ is also special. If $a = 0$ then $b > 0$ and $x = (x - 232) + 232$ with $x - 232$ special. Starting from $dp[0] = \text{True}$, the DP will mark all reachable values true, including x . So the equivalence holds, and backtracking from n correctly recovers a and b .

Runtime Analysis

We compute $dp[0..n]$ once, and each state checks at most two transitions, so runtime is $O(n)$. Backtracking subtracts at least 197 each step, so it is also $O(n)$. Space is $O(n)$.

Pseudo code

```
Special(n):
    dp[0] = True
    prev[0] = -1
    for x = 1 to n:
        dp[x] = False
        prev[x] = -1
        if x >= 197 and dp[x-197] == True:
            dp[x] = True
            prev[x] = 197
        else if x >= 232 and dp[x-232] == True:
            dp[x] = True
            prev[x] = 232

    if dp[n] == False:
        return "no solution"

    a = 0; b = 0; x = n
    while x > 0:
        if prev[x] == 197:
            a = a + 1
            x = x - 197
        else:
            b = b + 1
            x = x - 232
    return (a, b)
```

Final Answer (boxed)

DP reachability: $dp[x] = dp[x - 197] \vee dp[x - 232]$. If $dp[n]$ true, backtrack to output (a, b) . $O(n)$.

Problem 4: SubsetSum. **Input:** set $S = \{s_1, \dots, s_n\}$ of non-negative integers and target B . **Output:** subset $S' \subseteq S$ with sum B or **FALSE**. Give pseudocode that runs in $O(nB)$ time but uses only $O(B)$ extra space and returns the actual set S' .

Answer

Use a 1D DP array $dp[0..B]$ and update from B down to s_i for each item. To reconstruct the subset, store for each reachable sum t the last item index used and the previous sum before adding it.

Explanation

The classic DP uses $dp[i][t]$ for the first i items, which costs $O(nB)$ space. We can compress space because each row depends only on the previous row. Updating sums from right to left prevents reusing the same item twice. If we store a pointer each time we first make a sum reachable, we can follow pointers from B back to 0 to list the chosen items. This keeps extra space linear in B .

Formula

For each item s_i and each sum $t \geq s_i$ (updated in decreasing t):

$$dp[t] \leftarrow dp[t] \vee dp[t - s_i].$$

Proof of Correctness

Claim: At the end, $dp[t]$ is true iff some subset of $\{s_1, \dots, s_n\}$ sums to t , and the pointers reconstruct such a subset.

We prove by processing items in order. When processing item s_i , if $dp[t - s_i]$ was true before using s_i , then setting $dp[t]$ to true corresponds to taking that subset and adding s_i , which gives sum t . Updating t from B down to s_i ensures $dp[t - s_i]$ refers to the state before considering s_i , so each item is used at most once. If $dp[B]$ is true, following $(parent, prevSum)$ repeatedly subtracts chosen items and reaches 0, producing a valid subset summing to B . If $dp[B]$ is false, then no subset can form B because we explored all item choices under the recurrence.

Runtime Analysis

There are n items and we scan $t = 0..B$ for each, so time is $O(nB)$. Arrays $dp, parent, prevSum$ have size $B + 1$, so extra space is $O(B)$.

Pseudo code

```
SubsetSum1D(S[1..n], B):
    dp[0] = True
    for t = 1 to B:
        dp[t] = False
        parent[t] = -1
        prevSum[t] = -1

    for i = 1 to n:
        for t = B down to S[i]:
            if dp[t] == False and dp[t - S[i]] == True:
                dp[t] = True
                parent[t] = i
                prevSum[t] = t - S[i]

    if dp[B] == False:
        return FALSE

    Sprime = empty set
    t = B
    while t > 0:
        i = parent[t]
        add S[i] to Sprime
        t = prevSum[t]
    return Sprime
```

Final Answer (boxed)

1D DP updated right-to-left with pointers reconstructs S' . Time $O(nB)$, extra space $O(B)$.

Problem 5: ImportExport(A, B). **Input:** arrays $A[0..n-1]$ and $B[0..n-1]$ of length n with positive integers. **Output:** $\max_{j>i}(B[j] - A[i])$ or 0 if all are negative. Give (i) a recursive $O(n)$ algorithm and (ii) an $O(n)$ DP algorithm.

Answer

We maintain the minimum buy price seen so far in A and the best profit seen so far. At time j , the best profit by selling at j is $B[j] - \min(A[0..j-1])$. We update the best profit and also update the minimum buy price.

Explanation

For any fixed selling time j , the best buying time $i < j$ is the one with smallest $A[i]$ among earlier times. If we keep that minimum as we scan, then we can compute the best profit for each j in $O(1)$. Taking the maximum over all j gives the global best profit. This avoids checking all pairs (i, j) , which would be $O(n^2)$.

Formula

Let $\min A(j) = \min(A[0..j])$ and $\text{best}(j)$ be the best profit using sell time at most j :

$$\min A(0) = A[0], \quad \text{best}(0) = 0,$$

$$\min A(j) = \min(\min A(j-1), A[j]), \quad \text{best}(j) = \max(\text{best}(j-1), B[j] - \min A(j-1)).$$

Proof of Correctness

Claim: The algorithm returns $\max_{j>i}(B[j] - A[i])$ or 0 if all profits are negative.

For each j , the best profit that sells at j equals $B[j] - \min_{i<j} A[i]$, because subtracting the smallest possible earlier buy price maximizes the difference. The algorithm maintains $\min A(j-1) = \min_{i<j} A[i]$, so it correctly computes the best sell-at- j profit. By keeping $\text{best}(j)$ as the maximum over all profits seen so far, it equals the best profit for all pairs with sell time $\leq j$. At the end, $\text{best}(n-1)$ is the maximum profit over all valid pairs; initializing with 0 ensures we return 0 if all differences are negative.

Runtime Analysis

We do one pass with $O(1)$ work per index, so time is $O(n)$ and space is $O(1)$.

Pseudo code (Recursive)

Pseudo code

```
Solve(j): # returns (minSoFar, bestProfit) for prefix 0..j
    if j == 0:
        return (A[0], 0)
    (m, best) = Solve(j-1)
    newBest = max(best, B[j] - m)
    newMin = min(m, A[j])
    return (newMin, newBest)

ImportExport_Rec(A,B):
    (_, ans) = Solve(n-1)
    return ans
```

Pseudo code (DP)

Pseudo code

```
ImportExport_DP(A,B):
    minSoFar = A[0]
    bestProfit = 0
    for j = 1 to n-1:
        bestProfit = max(bestProfit, B[j] - minSoFar)
        minSoFar = min(minSoFar, A[j])
    return bestProfit
```

Final Answer (boxed)

Maintain $minSoFar = \min(A[0..j-1])$ and update $best = \max(best, B[j] - minSoFar)$. Time $O(n)$.

Challenge Yourself: Minimum swaps to sort a,b,c

Answer

Count how many a 's, b 's, and c 's there are, which fixes the target segments for each letter in the sorted string. Count mismatches $M_{x,y}$, meaning how many letter x are currently in the segment that should be y . First do all direct swaps between pairs of mismatches (like swapping an a in the b -segment with a b in the a -segment). The remaining mismatches form 3-cycles, and each 3-cycle needs exactly 2 swaps.

Explanation

In the final sorted string, every position belongs to exactly one of three segments. A direct swap can fix two wrong positions at once if each holds the other's correct letter. After we do all possible direct swaps, every remaining wrong letter must be part of a cycle $a \rightarrow b \rightarrow c \rightarrow a$ (or the reverse), because there are only three types. A 3-cycle cannot be fixed by one swap, but it can always be fixed by exactly two swaps. So counting direct swaps plus twice the number of remaining 3-cycles gives the minimum.

Formula

Let $M_{x,y}$ be the number of x letters in the y segment. Direct swaps:

$$s_{ab} = \min(M_{a,b}, M_{b,a}), \quad s_{ac} = \min(M_{a,c}, M_{c,a}), \quad s_{bc} = \min(M_{b,c}, M_{c,b}).$$

Remaining misplacements:

$$R = (M_{a,b} - s_{ab}) + (M_{a,c} - s_{ac}) + (M_{b,c} - s_{bc}),$$

so number of 3-cycles is $R/3$ and extra swaps are $2(R/3)$. Total:

$$s_{ab} + s_{ac} + s_{bc} + 2 \cdot \frac{R}{3}.$$

Proof of Correctness

Claim: The computed value equals the minimum number of swaps.

Each direct swap fixes two positions and cannot be improved because those two letters must eventually move to each other's segments, so doing $\min(M_{x,y}, M_{y,x})$ direct swaps is optimal for that pair. After all direct swaps, every remaining wrong letter must be in a 3-cycle because there are no remaining opposite mismatches to swap. Any 3-cycle needs at least 2 swaps since one swap can fix at most two letters and a 3-cycle has three misplaced letters. We can always fix a 3-cycle in 2 swaps by swapping one element into its correct segment and then swapping the remaining element into place. Therefore direct swaps plus 2 per cycle is optimal and achievable.

Runtime Analysis

We compute counts and mismatches in one scan, so runtime is $O(n)$ and space is $O(1)$.

Pseudo code

```
MinSwapsABC(S[1..n]):  
    count na, nb, nc  
    define segments: A=1..na, B=na+1..na+nb, C=rest  
    compute mismatch counts M[x,y] by scanning S  
  
    sab = min(M[a,b], M[b,a])  
    sac = min(M[a,c], M[c,a])  
    sbc = min(M[b,c], M[c,b])  
  
    R = (M[a,b]-sab) + (M[a,c]-sac) + (M[b,c]-sbc)  
    return sab + sac + sbc + 2*(R/3)
```

Final Answer (boxed)

Minimum swaps = $s_{ab} + s_{ac} + s_{bc} + 2(R/3)$ computed from mismatch counts. Time $O(n)$.